Umbraco.Automate — Functional Overview

What is it?

Umbraco.Automate brings Zapier/n8n-style automation directly into Umbraco CMS. Editors, developers, and AI agents can build event-driven workflows without leaving the backoffice — no external automation platforms, no custom code.

It is the first CMS-embedded automation engine in the .NET ecosystem.

Why?

Problem	How Automate solves it
Umbraco users rely on external platforms (Zapier, Power Automate) or custom code for automation	Built-in automation engine, native to the backoffice
External platforms require sending CMS data to third parties	Data stays in-house — triggers fire from Umbraco events natively
No way to connect DXP products (Forms, Commerce, Workflow, Engage) through automation	Shared automation bus — each product contributes triggers and actions
AI agents have no way to orchestrate multi-step processes	Bidirectional AI integration — agents as actions, automations as agent tools
Audit and governance are bolted on, not built in	Full audit trail from day one — every run, every step, every input/output

---

Core Concepts

Concept	Description
Automation	A user-defined workflow — a trigger plus a sequence of steps, built visually on a canvas
Trigger	The event that starts an automation (e.g. "Content Published", "Webhook Received", "Scheduled")
Action	A reusable unit of work within an automation (e.g. "Send Slack Message", "Publish Content", "HTTP Request")
Step	A configured instance of an action within an automation, with specific settings and position on the canvas
Connection	A named, reusable set of credentials for an external service (e.g. "Production Slack", "Staging SMTP")
Run	A single execution of an automation, with full step-by-step audit trail
Expression	Data binding between steps using `${ trigger.contentName

---

What can you automate?

Built-in (ships with the product)

Group	Triggers	Actions
Core	Manual, Scheduled (CRON), Webhook Received	HTTP Request, Delay, Log Message, Set Variable

Content	Content Published, Unpublished, Saved, Deleted, Moved	Publish, Unpublish, Create, Update, Delete Content
Media	Media Uploaded, Media Deleted	Upload Media, Delete Media
Members	Member Created, Approved, Locked Out	Create Member, Update Member, Assign Group

DXP integration packages

Package	Triggers	Actions
Umbraco.Automate.Forms	Form Submitted, Form Entry Approved	Submit Form, Export Entries
Umbraco.Automate.Commerce	Order Placed, Order Status Changed, Payment Captured, Stock Low	Update Order Status, Send Order Email, Adjust Stock
Umbraco.Automate.Workflow	Approval Requested, Completed, Rejected	Request Approval, Approve Content, Reject Content
Umbraco.Automate.Engage	Segment Entered/Exited, Persona Assigned	Assign Persona, Add to Segment, Trigger Personalization

Third-party / community extensibility

Developers can create custom triggers and actions as NuGet packages. Examples:

- Slack: Send Message, Create Channel
- Email (SMTP): Send Email
- AI: Generate Text, Classify Content, Summarize

Visual Canvas Editor

Automations are built in a full-screen visual node graph editor within the Umbraco backoffice.

- **Drag-and-drop** node placement with connection lines between steps
- **Add button** opens a categorised picker modal to insert triggers/actions
- **Pencil icon** on each node opens a settings modal with auto-generated forms
- **No sidebars** — maximum canvas space for complex automations
- **Minimap, keyboard shortcuts, copy/paste, undo/redo**
- **Dry run mode** — test an automation visually with green/red status per node

---

Data Flow Between Steps

Steps pass data to each other using an expression syntax inspired by Umbraco's UFM:

```
${ trigger.contentName }                      — reference trigger output
${ steps.sendEmail.messageId }                — reference a previous step's output
${ trigger.body | truncate:100 }              — apply a filter
${ trigger.publishedDate | formatDate:yyyy-MM-dd }  — format a date
${ trigger.email | fallback:no-reply@example.com }  — provide a default value
```

Filters are chainable (`| stripHtml | truncate:200:...`) and extensible — developers can register custom filters.

Versioning & Publishing

Automations follow a **draft → publish** lifecycle, consistent with Umbraco's content model:

Status	Triggers fire?	Description
Draft	No	Being edited. Can be tested via dry run.
Published	Yes (if enabled)	Live version responds to triggers.
Inactive	No	Explicitly deactivated. Published version retained for rollback.

- **Editing a published automation** saves draft versions without affecting the live version
- **"Unpublished changes"** indicator shows when the draft is ahead of the published version
- **Every save** creates an immutable version snapshot
- **Rollback** to any previous version at any time
- **In-flight runs** always complete on the version they started with — publishing a new version does not disrupt running automations
- **Kill switch** — `IsEnabled` toggle for emergency disable without losing published state

---

Governance & Observability

Audit Trail

Every automation run produces a complete record:

- Exact input/output data at each step
- Duration, retry count, error details
- Who or what initiated the run (user, system event, AI agent, webhook)
- Which automation version was executing

Run Explorer

A dedicated backoffice view for investigating runs:

- **Run list** — filterable by automation, status, date range, initiator
- **Run detail** — visual replay on the same canvas with step-by-step status overlay
- **Error drill-down** — stack traces, retry history, the exact step that failed
- **Duration metrics** — per-step and total run timing

Safety Features

Feature	Description
Dry run mode	Execute without side effects — steps return what they *would* do
Rate limiting	Configurable max runs per automation per time window
Kill switch	Global and per-automation emergency disable
Version rollback	Restore any previous version instantly
Sensitive data masking	Credentials encrypted at rest, masked in run logs
Timeout enforcement	Per-step timeouts prevent runaway actions
Trigger deduplication	Prevents duplicate runs from duplicate events

Failure Notifications

When a run fails, configurable notification channels alert the right people:

- **Backoffice** — toast/badge for logged-in users (always on)
- **Email** — failure details to configured recipients
- **Webhook** — JSON payload to Slack, Teams, PagerDuty, etc.
- **Extensible** — developers can add custom notification channels

Each automation configures its own notification preferences — which channels, who to notify, and on what conditions.

Access Control

Leverages Umbraco's existing user group / permission model:

- View automations (read-only)
- Edit automations (create, modify, enable/disable)
- Execute automations (manually trigger)
- Administer automations (delete, manage all users' automations)

Connections (Credential Management)

External service credentials are managed as **named connections** — reusable, environment-specific, and separate from automation definitions.

- Steps reference a connection by ID (e.g. "Slack Notifications")
- Connections store credentials encrypted at rest
- Connections are **environment-specific** — they never transfer between environments via Deploy
- Multiple automations can share the same connection
- Connections are versioned for audit trail and rollback

Incoming Webhooks

Umbraco's built-in webhook system is outgoing only. Automate adds **incoming webhook support**:

- Each automation with a "Webhook Received" trigger gets a unique URL
- Optional shared secret / HMAC signature validation
- Full request captured (headers, body, query string) as trigger data for downstream steps
- Configurable allowed HTTP methods

Human-in-the-Loop (HITL)

Critical for AI integration and content governance:

- **"Request Approval" action** — suspends the automation and notifies approvers
- **Approval UI** in the backoffice — approve / reject / request changes with comments
- **AI agent gate** — steps can require human review before an AI agent's proposed action executes
- **Configurable** — which steps require approval can be set per-automation or globally
- **Integration with Umbraco Workflow** — automations can participate in existing content approval processes

---

AI Integration (via Umbraco.AI)

All AI capabilities come from Umbraco.AI. The integration is bidirectional:

AI → Automate: Agents as Actions

Automations can execute AI agents as steps:

- Content Published → Execute "Translate Agent" → Publish translated variants
- Form Submitted → Execute "Lead Scoring Agent" → Route to sales or nurture
- Scheduled → Execute "Content Audit Agent" → Email report to editors

AI → Automate: AI Events as Triggers

AI lifecycle events can start automations:

- Agent Failed → Send Slack alert
- Agent Completed (content generation) → Start content review workflow

Automate → AI: Automations as Agent Tools

Automations can be exposed as tools that AI agents can invoke:

- Agent sees a tool called "publish-campaign-content"
- Invoking it triggers a multi-step automation
- The agent doesn't need to know the individual steps

AI governance: Configurable option to require human approval before any AI-initiated automation executes.

Umbraco Deploy Support

What transfers

- Automation definitions (name, steps, connections, canvas layout)
- Folder structure
- Non-sensitive step settings

What does NOT transfer

- Credentials (stripped, must be configured in target environment)
- Run history
- Enabled state (arrives disabled — must be explicitly published)

How credentials work across environments

- Steps reference connections by ID internally
- During Deploy export, IDs are swapped to aliases
- During import, aliases are resolved back to IDs in the target environment
- Missing connections are flagged immediately — not discovered at runtime when a run fails

---

Backoffice UI

Section: Automations

A new backoffice section with:

- **Tree** — automations organized in folders (like Data Types), with drag-and-drop, nested folders, and context menus
- **Dashboard** — status cards (broken automations, failing automations, stuck runs, pending approvals), recent activity, quick stats
- **Automation Editor** — two workspace apps:
  - **Workflow tab** — the visual canvas editor with publish/save-draft split button
  - **Info tab** — version history with rollback, entity metadata (ID, alias, status, dates)
- **Run Explorer** — filterable run list with visual graph replay and step-by-step data inspection
- **Settings panel** — global settings, registered triggers/actions catalogue, connection management

---

Extensibility for Developers

- **Custom triggers and actions** — implement an interface, add an attribute, register via DI
- **Custom expression filters** — extend the `${ }` expression system with new filters
- **Custom notification channels** — add PagerDuty, OpsGenie, or any alerting system
- **Middleware pipeline** — insert cross-cutting concerns (logging, metrics, validation) around action execution
- **Lifecycle notifications** — react to automation events (saving, running, completing) from other packages
- **Project template** — `dotnet new umbraco-automate-actions` scaffolds a new provider package
- **Test harness** — unit test actions in isolation without running the full engine

---

Database & Infrastructure

- **SQL Server and SQLite** supported (same as Umbraco CMS)
- **Shared database by default** — tables coexist with Umbraco, prefixed with `UmbracoAutomate_`
- **Separate database optional** — configure `umbracoAutomateDbDSN` connection string (follows Umbraco Commerce convention)
- **Distributed deployment** — swap in Redis, Azure, RabbitMQ, or AWS queue/lock providers via NuGet packages
- **Health checks** — engine status, queue depth, and data retention registered with Umbraco's health check system
- **OpenTelemetry** — metrics for runs, failures, and step duration (Prometheus, Application Insights, etc.)
- **Data retention** — configurable purge of old run data, per-automation or globally

---

Phased Delivery

Phase 1: Foundation (MVP)

Core engine, basic triggers/actions (Manual, Scheduled, Webhook, Content Published), canvas editor, run logging, security hardening, draft/publish lifecycle.

Phase 2: HITL, Branching & Hardening

Approval workflows, conditional branching (If/Switch), named connections with OAuth2, Deploy integration, failure notifications, dry run mode, import/export, automation templates.

Phase 3: AI Integration

AI agents as actions, AI events as triggers, automations as AI tools, HITL gates for AI-initiated runs.

Phase 4: DXP Providers

Forms, Commerce, Workflow, and Engage integration packages.

Phase 5: Advanced Features

Parallel execution, sub-automations, version diff, audit log immutability, distributed tracing.

Key Risks

Risk	Mitigation
WorkflowCore (execution engine) has a single maintainer	MIT-licensed, small codebase — practical to fork if needed. Clean abstraction layer means engine is swappable.
Performance at scale	Distributed execution supported. Data retention purge. Queue depth limits.

Credential exposure	Encrypted at rest, masked in logs, stripped from Deploy transfers
Duplicate runs from duplicate events	Trigger deduplication with configurable idempotency window
Third-party actions referencing entities without Deploy support	Pre-transfer validation blocks transfers with clear error messages

---

Success Criteria

1. **Editors** can create a "Content Published → Send Slack Message" automation without developer help
2. **Developers** can create a custom trigger and action in under an hour
3. **Every run** has a complete audit trail — inputs, outputs, errors, duration, initiator
4. **AI agents** can trigger automations and have their actions reviewed by humans before execution
5. **DXP products** can contribute triggers and actions without depending on each other